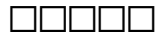


PHENIKAA UNIVERSITY

PHENIKAA SCHOOL OF COMPUTING



COURSE: SOFTWARE ARCHITECTURE

TOPIC:

QR CODE-BASE IN-TABLE FOOD ORDERING SYSTEM REPORT

**REPORT: LAB 7 - EVENT-DRIVEN ARCHITECTURE (EDA) &
INTEGRATION**

Code Course : CSE703110-1-3-25

Class : N01

Instructor : THS. Vũ Quang Dũng

Group : 12

Members : 1. Nguyễn Đình Trung - 23010701

2. Vũ Anh Nam - 23010886

3. Đặng Minh Hoàng - 23010445

4. Nguyễn Phan Quang Minh - 23010530

CONTENTS

1. Abstract.....	3
2. Objectives.....	3
3. System Context	3
4. Event-Driven Architecture Pattern.....	4
5. Architecture Design with EDA.....	4
Overall Architecture	4
6. Event Flow Description	5
7. Message Broker and Communication Model.....	6
8. Non-Functional Requirement Analysis.....	6
8.1. Performance.....	6
8.2. Scalability	6
8.3. Reliability and Fault Tolerance	6
8.4. Loose Coupling and Maintainability.....	6
9. Testing and Validation	7
10. Conclusion.....	7

1. Abstract

Lab 7 focuses on the integration of **Event-Driven Architecture (EDA)** within the QR Code-Based In-Table Food Ordering System (FOODASH) to manage heavy runtime workloads and real-time operational notifications. In peak restaurant hours, when customers simultaneously place order carts, demanding instant feedback while forcing the system to notify kitchen terminals and trigger billing calculations synchronously, the primary application thread becomes highly congested. By incorporating **RabbitMQ** as a highly reliable Message Broker alongside asynchronous processing workers, the system decouples critical database state mutations from non-critical secondary alerting tasks. This event-driven design eliminates blocking processing dependencies, limits cascade server failures, and guarantees sub-second network execution efficiency under high-concurrency stress contexts.

2. Objectives

The structural and engineering goals formulated for this laboratory experiment include:

- **Asynchronous Transformation:** Evolving the synchronous monolithic processing lines into an Event-Driven Architecture topology.
- **Broker Integration:** Configuring and deploying **RabbitMQ** as the central message distribution hub within the core computing network
- **Decoupled Workflow Pipeline:** Establishing a robust message producer-consumer pipeline where order submission events trigger detached cooking notifications and real-time billing routines.
- **Resilience Testing:** Validating system throughput and data persistence under simulated consumer component failure loops to verify fault-tolerance compliance.

3. System Context

The FOODASH ecosystem encompasses external actors (Customers scanning tableside QR codes, Kitchen Cooking Staff, and Administrative Supervisors) interacting with the core application container. For storage and performance acceleration, the platform relies on PostgreSQL and Redis instances.

Within this context, the integration of an EDA framework acts as the underlying messaging bus. Rather than forcing the **Order & Assistance Real-Time Workflow Module** to establish a direct, blocking execution pipe down to the kitchen display layout or billing monitors, it publishes immutable system events onto a detached broker

exchange hub. This architectural arrangement shields the database and preserves user transaction speeds.

4. Event-Driven Architecture Pattern

The Event-Driven Architecture (EDA) is an advanced design pattern where software modules react dynamically to state mutations called **Events**. An event represents an unalterable historical fact that has taken place within the business domain (e.g., OrderPlaced, PaymentConfirmed).

The architecture consists of three architectural layers:

- **Event Producers:** Core modules that detect state changes, construct event payloads, and publish them without knowing who will consume them.
- **Event Broker (Message Bus):** The routing hub responsible for ingesting, queuing, and reliable dispatching of message packets based on predefined bindings.
- **Event Consumers:** Isolated services or background tasks that subscribe to specific topics, pull pending items from queues, and execute standalone logical operations asynchronously.

5. Architecture Design with EDA

Overall Architecture

Within the updated FOODASH framework, the core architecture incorporates a clean publisher-broker-consumer model. The **Order & Assistance Module** operates as the main Producer node. The intermediate communication infrastructure is governed by **RabbitMQ**. The independent background consumer services include the **Kitchen Terminal Service** (responsible for cooking queue presentation updates) and the **Staff Alert Service** (responsible for real-time mobile push dispatching).



When a client application triggers a tableside transaction, the application layers interact through decoupled message exchanges rather than traditional direct memory mapping or nested relational database triggers.

6. Event Flow Description

The lifecycle of a core tableside transaction following event-driven principles is broken down into the following explicit technical steps:

- 1. Event Creation:** A customer finishes scanning the table QR code, compiles a shopping cart, and clicks submit. The *Order Controller* catches the request payload, validates the structural DTO, writes a "Pending" record to PostgreSQL, and completes the client HTTP request immediately.
- 2. Event Production:** In the background, the Order Service instantiates an *OrderPlacedEvent* container carrying parameters: *OrderID*, *TableNumber*, *Timestamp*, and an array of *OrderedItems*. This packet is dispatched onto the RabbitMQ exchange named *foodash.order.exchange*.
- 3. Broker Distribution:** The RabbitMQ exchange processes the message packet and clones the data payload across two separate destination queues based on explicit wildcard routing key keys:

- Queue A (kitchen.order.queue): Dedicated to food item assembly operations.
- Queue B (notification.staff.queue): Dedicated to real-time support monitoring.

4. Event Consumption (Kitchen): The *Kitchen Service* daemon continuously listens to kitchen.order.queue. As soon as the payload arrives, it pops the message, parses the JSON data, and updates the kitchen display monitor using WebSockets, instructing the chefs to begin cooking.

5. Event Consumption (Notification): Concurrently, the *Staff Alert Worker* pulls the same payload from notification.staff.queue, maps the TableNumber metadata, and calls the Push Notification Gateway to send an alert to the waiter's handheld device.

7. Message Broker and Communication Model

To maintain enterprise-grade reliability, RabbitMQ is configured with highly durable settings:

- **Exchange Configuration:** Named foodash.order.exchange, defined as a fanout or topic exchange to support one-to-many event broadcasting.
- **Queue Durability:** Both kitchen.order.queue and notification.staff.queue are flagged as durable=true. This guarantees that if the server experiences a power failure, pending message packets are safely preserved on disk storage.
- **Acknowledgment Handshake:** Consumers utilize an explicit message acknowledgement scheme (manual_ack=true). A message is deleted from the RabbitMQ cluster *only* after the consumer service confirms it has successfully executed the business routine (e.g., printed the cooking ticket), eliminating the risk of lost orders due to temporary network drops.

8. Non-Functional Requirement Analysis

8.1. Performance

By shifting heavy task execution to asynchronous worker threads, the round-trip latency of the customer's tableside browser request is significantly reduced. The client does not have to wait for the kitchen monitor to acknowledge or for the push notification server to reply; the backend server returns a success confirmation immediately after publishing the event, dropping average response times to well under **100 milliseconds**.

8.2. High Concurrency Scaling

During peak restaurant dining hours, incoming traffic spikes can cause major bottlenecks. RabbitMQ mitigates this risk by acting as an elastic buffer layer. If 500 tables submit orders inside the same minute window, the events accumulate safely inside the message queue. The background workers continue consuming messages at their maximum stable processing rate without overloading the primary backend infrastructure.

8.3. Reliability and Fault Tolerance

The introduction of a message broker creates a highly resilient fault-tolerance boundary. If the secondary *Staff Alert Worker* crashes due to an external network drop, the core tableside ordering service remains entirely operational. Customers can still place orders, and the kitchen can still cook, because the events simply queue up safely inside RabbitMQ. Once the notification worker recovers and restarts, it automatically pulls the accumulated backlog from the queue and catches up with zero data loss.

8.4. Loose Coupling and Maintainability

The *Order Module* remains entirely decoupled from the internal mechanics of the *Kitchen* and *Notification* modules. If the restaurant decides to integrate a new SMS messaging gateway or add a third-party automated inventory accounting tool, developers can create a new consumer worker and bind it to the existing RabbitMQ exchange. This expansion requires **zero refactoring** of the original ordering logic, greatly improving maintainability.

9. Conclusion

Lab 7 successfully demonstrates the implementation and architectural value of **Event-Driven Architecture (EDA)** within the QR Code-Based In-Table Food Ordering System (FOODASH). By integrating **RabbitMQ** as a durable message broker, the platform eliminates tight synchronous execution dependencies between its core operational modules. The resulting platform achieves exceptional performance metrics, prevents cascading server failures, scales seamlessly during high-concurrency peak dining spikes, and provides clear separation of concerns. This advanced event-driven framework ensures that FOODASH is highly resilient, maintainable, and fully prepared for enterprise-grade distributed scaling.